

OOP Summary

ايه هيا ال OOP ؟

الـ OOP هي طريقة موحده لكتابة البرامج بحيث نقسم البرنامج إلى

Objects

عشان يسهل فهم الكود بين المطورين المختلفين ويسهل التعديل عليه

وهكذا

فائدة استخدام OOP

الـ OOP بتخلي البرمجة:

1 أسهل في تنظيم الكود

بدل ما الكود يكون كبير ومتلخبط.

2 أسهل في الفهم

لأن البرنامج بيبقى مقسوم لأجزاء واضحة.

3 أسهل في التعديل

لو عايز تعدل حاجة بتعدل جزء معين فقط.

4 إعادة استخدام الكود

تقدر تستخدم نفس الكود أكثر من مرة.

Class

يعني ايه Class ؟

الـ **Class** هو قالب أو تصميم بنستخدمه علشان ننشئ منه Objects.

يعني هو الوصف العام للشيء وليس الشيء نفسه.

مثال:

لو عندنا **Class** اسمها **Car**

هيكون فيها:

• color

• speed

وظائف مثل:

• move()

• stop()

يعني ايه Object ؟

الـ **Object** هو نسخة حقيقية من الـ **Class**.

يعني لما نستخدم الـ **Class** وننشئ منها كيان في البرنامج.

مثال:

لو عندنا **Class** اسمها **Car**

نقدر نعمل Objects مثل:

• → car1 لونها أحمر

• → car2 لونها أزرق

كلهم من نفس الـ **Class** لكن بياناتهم مختلفة.

\$ قبل الدخول في ال OOP في طريقه للكتابة عشان ال **Clean Code** وتسهيل فهم الكود بالشكل دا

(1) Class 1 Word : Student

(2) Class 2 Word : StudentSchool

(3) Attribute 1 Word : id

(4) Attribute 2 Word : idStudent

(5) Attribute 3 Word : idStudentSchool

Constructors

هو ميثود خاص يستخدم لعمل تهيئة (Initialization) للأوبجكت لما بيتعمله إنشاء .

اسمه لازم يكون نفس اسم الكلاس .

ملوش نوع رجوع (No return type) حتى مش void.

بيتنادي عليه تلقائيًا أول ما نعمل object

```
class Student {  
    String name;  
    int age;  
  
    // Constructor  
    Student(String n, int a) {  
        name = n;  
        age = a;  
    }  
}
```

Encapsulation

الهدف منه هو حماية البيانات ومنع الوصول المباشر ليها.

يتم عن طريق اننا نخلي المتغيرات `private` وبعدها نستخدم `setter and getter` عشان نتحكم في الوصول للبيانات والتعديل عليها ووضع شروط وقواعد علي البيانات عكس انها تبقي `public` واي حد يقدر يعدل عليها او انها تبقي `private` بس فا محدش خالص يقدر يعدل عليها لا هيا `private` بس مع وضع شروط خاصه ليها عن طريق ال `setter and getter`

```
class Person {  
    private String name;  
  
    // Getter  
    public String getName() {  
        return name;  
    }  
  
    // Setter  
    public void setName(String n) {  
        name = n;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Person p = new Person();  
        p.setName("Ali"); // تعديل القيمة  
        System.out.println(p.getName()); // قراءة القيمة  
    }  
}
```

Inheritance

هو مفهوم في الـ OOP بيخلي كلاس يرث (ياخد) خصائص وميثودز من كلاس تاني

بدل ما اعمل كلاسات كتير فيها كذا ميثود بتادي نفس الوظيفة
اعمل كلاس واحده بس فيها الكلام دا واورثها للكلاسات الي هتشتغل
بالكلام دا توفيراً للوقت وان الكود يكون انضف.

الكلاس الي بيورث اسمه Parent (Superclass)
واللي بيورث منه اسمه Child (Subclass)

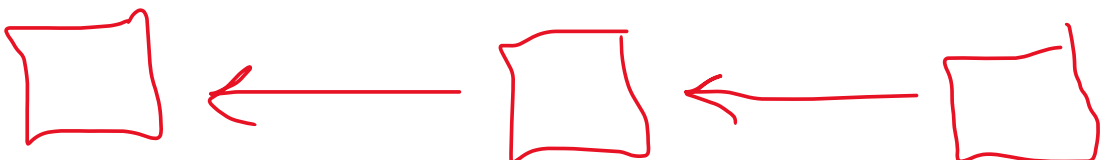
```
class Animal {  
    void eat() {  
        System.out.println("This animal eats");  
    }  
}  
  
class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog is barking");  
    }  
}
```

والكلاس بيورث من كلاس واحد

مينفعش يورث من كذا كلاس

في حاله اننا عايزين نحقق معادله زي دي
بنخلي كلاس يورث كلاس

والي ورث دا يورث للتاني بحيث انه جمع كل الصفات الي عايزين
نورثها بدل ما نعمل 2 extend بالشكل دا



Polymorphism

في ال **inheritance** الكلاس الاب بتورث كل الفانكشنز الي فيها لباقي الكلاسات الي عامله **extend** واي وظيفه بتعملها الفانشكنز دي بيورسوها من الكلاس الاب طيب افرض عايزين كلاس من الكلاسات الي هنزرسها من الاب نغير وظيفتها مع الحفاظ علي مفهوم الوراثة بس انا عايز فنكشن معينه هيا الي تتغير والباقي تمام

* مثال : عندي الكلاس الاب بتشتغل ب OracleDB والمفروض كله يورث الكلام دا بس مثلا في كلاس معينه عايزها انها تشتغل بي MySQL بدلا من اوراكيل

هنا بقا يجي مفهوم ال **Polymorphism**

وبيتم عن طريق اننا مروح للكلاس الي الفانكشن فيها هتبقى مختلفه ونكتب فيها نفس ال Heder بتاع الفانكشن الي في الاب مسبوقة بي

@Override

بالشكل دا

```
3 usages
@Override
public void printData(){
    System.out.println("data of Student");
}
```

نفس Header الفانكشن الي في الكلاس الاب لازم يبغي زيه بالظبط

لازم نتكتب قبل ما نعمل Override

العملية الي بتتعمل في الكود اسمها **Override**
الحاله دي بقا بشكل هام بيتقال عليها ال **Polymorphism**

في حاله عايزين نعمل العكس
اوننا نجبر كل الكلاسات انها تورث الفانكشن ومتقدرش تعمل **Overrid**
هنستخدم مصطلح قبل اسم الفانكشن اسمه **Final**
بالشكل دا

```
public final void printData(){  
    System.out.println("data of BaseEntity");  
}
```

Abstract Class

المفروض كل كلاس نقدر ناخذ منه **Object** طب لو في كلاس مش
عايزين ناخذ منه **Object** زي كلاس مخصصه انها تورث لكلاسات
تانيه بس وهكذا
هنا يجي دور ال **Abstract Class** وهو اسم الكلاس مسبق بكلمه
Abstract بالشكل دا

```
public abstract class BaseEntity {  
  
    2 usages 1 override
```

وكدا متعرفش تاخذ **Object** في ال **Main** من الكلاس وهيظهر ايرور
بالشكل دا

```
BaseEntity baseEntity = new BaseEntity();  
baseEntity.printData1(); // data of BaseEntity
```

ERROR

الـ **Abstract Class** يمنع أي حد يكرت منه اوبجكت
بس بيسمع انك تكرت منه **Abstract Function**

ايه هيا الـ **Abstract Function**
هيا فانكشن ليها **Header** بس ملهاش **Implementaion**
يعني ملهاش كود جوا الاقواس بالشكل دا

```
public abstract void printData3();  
}
```

ووظيفتها انها علي عكس الفانكشن العاديه
دي بتجبرك انك لازم تاخدها وتعمل عليها **Override**

يبقي كدا عرفنا لحد دلوقتي 3 أنواع **Functions**

1 فانكشن عاديه تقدر تعمل عليها **Override**

```
public void printData1(){  
    System.out.println("data of BaseEntity");  
}
```

2 فانكشن متقدرش تعمل عليها **Override** باستخدام **Final**

```
public final void printData2(){  
    System.out.println("data of BaseEntity");  
}
```

3 فانكشن بتجبرك تعمل عليها **Override** باستخدام **Abstract**


```
public abstract void printData3();
```

Final Class

هو عكس ال Abstract Class

اقدر اكريت منه Object بس مقدرش اعمل منه Extend

```
public final class BaseEntity {
```

لو جربت تعمل Extend هيدي ايرور



```
public class Student extends BaseEntity {
```

Interface

هو Abstract Class بس بشؤوط معينه وهي :

ان كل ال Mesthods الي فيه تكون Abstract

مش زي ال Abstract Class العاديه تقدر تحط فيها Methods

عاديه او Final عادي لا دي Abstract Classes بس

```
public interface BaseEntityService {  
    void printData1();  
    void printData2();  
    void printData3();  
    void printData4();  
    void printData5();  
}
```

في الكلاسات الثانيه بتعمل Extend لان في حاجات جوا الفانكشنز تورثها

اما بالنسبه للInterface فا كلها Abstract Classes
مش معمولها Implementaion يعني كلها فانكشنز فاضيه

فا لما تيجي تورث بتتحول من مصطلح **Extend** لي **implements**

```
public class Student implements BaseEntity {
```

الخلاصه ؟

كلاس مع كلاس : **extend**

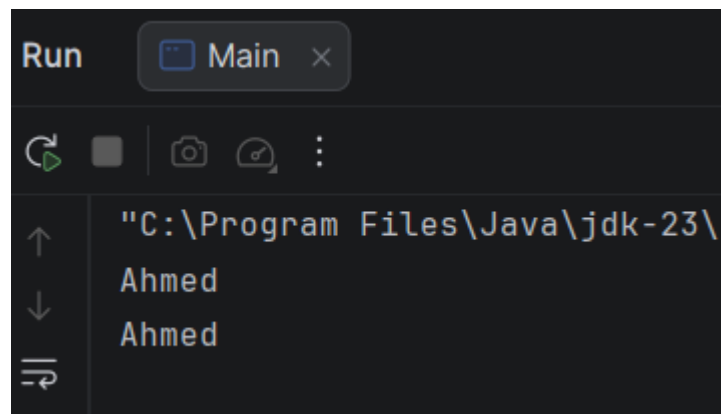
كلاس مع انترفيس : **implements**

Static Attribute

وهيا ان ال attribute يبقى **shared** بين كل ال **Objects**

```
public class Main {  
    public static void main(String[] args){  
  
        BasicEntity basicEntity1 = new BasicEntity();  
        basicEntity1.id = 10;  
        basicEntity1.name = "Ahmed";  
  
        BasicEntity basicEntity2 = new BasicEntity();  
    }  
}
```

```
public class BasicEntity { 4 usages  
  
    public int id; 1 usage  
  
    public static String name;  
}
```



زي ما موضح في الصور

2 objects Name attribute يبقى **shared** في كل من ال

مع ان بدون ال **Static** المفروض 2 baseEntity تبقي بي **null**

بس كدا أي تعديله تحصل في ال attribute هتسمع في كل ال **objects**

وكدا ال attribute يبقى **shared**

وكدا نقدر ننده للattribute دايركت عن طريق اسم الكلاس
علي طول وخلص بالشكل دا

```
BaseEntity baseEntity1 = new BaseEntity(); // [id = 1, name = "ahmed"]
baseEntity1.id = 1;
BaseEntity.name = "ahmed";

BaseEntity baseEntity2 = new BaseEntity(); // []
```

ملوش لازمه ناخذ اوبيجت من الكلاس وبعد كدا ننده للatribوت لما هز
كدا كدا shared

Static Method

هيا method عاديه لكن مش بتتعامل الا مع static attribute
بالشكل دا

```
public static String name;

2 usages
public static void start(){
//      System.out.println("id : " + id);
      System.out.println("name : " + name);
}
}
```

Final Attribute

```
2 usages 1 related problem
public final int id = 5;
}
```

دا عباره عن attribute قيمته مش بتتغير
زي في الصوره الattribute دا قيمته لازم تفضل 5 ثابتة مينفعش تغيرها

تكملة ال Interface

أي attribute في الانترفيس بياخذ صفتين (static and final)

```
public interface SchoolService {  
    int id = 5; // static final
```

Default in interface

المعروف ان في الميثود في الانترفيس مش بيبيقي ليها body
بس في طريقه نخلي ليها body وهيا عن طريق ال default

```
default boolean removeStudent() {  
    System.out.println("removeStudent..");  
}
```

في الشكل دا كذا بقا في initial implementation للميثود دي

Static attribute

```
1 usage  
static boolean getStudent() {  
    System.out.println("getStudent..");  
    return true;  
}
```

منقدرش نعمل عليه Override
ونقدر نستدعيه في ال Main

مينفعش الكلاس يورث من 2 كلاس
بالشكل دا

```
public class Student extends BaseEntity, PublicData {  
  
}
```

لكن لو ال 2 كلاس عباره عن intrtface
ينفع يتعمله كذا implement عادي

```
public class Student implements BaseEntity, PublicData {  
  
}
```